

EXPERIMENT 8

COLLECTIONS

8).A)

AIM:

Consider A patient class and assume the necessary fields such as disease or problem, name, address, phno, gender, age. Consider a collection of Patient Object and do the following operation.

- i) List the Patient details of a particular gender
- ii) List the Patient name and disease who are greater than particular age
- iii) Display the patient name of a particular patient ID.
- iv) Sort and display the whole patient detail in thier alphabetically order.

PROCEDURE:

1. **Start**
2. **Create Patient Class:**
 - Define a class with private fields for patient details (ID, name, disease, address, phone number, gender, age).
 - Add a parameterized constructor and getter methods for all fields.
 - Override the toString method for a formatted patient information display.
3. **Main Logic:**
 - Use the PatientCollection class with a main method to manage patient data.
4. **Input Patient Details:**
 - Create a List<Patient> to store patient objects.
 - Use a loop to collect details of 5 patients (ID, name, disease, address, phone number, gender, age) from the user.
 - Instantiate Patient objects with the input data and add them to the list.
5. **Filter Patients by Gender:**
 - Ask the user to input a gender and display patients matching that gender.
6. **Filter Patients by Age:**
 - Prompt the user for an age.
 - List patients older than the specified age, showing their name and disease.
7. **Search by Patient ID:**
 - Ask for a patient ID and search the list for a matching ID.
 - Display the patient's name if found, or a message indicating no match.
8. **Sort Patients by Name:**
 - Use Collections.sort with a comparator to sort the list of patients alphabetically by their name.
 - Print the sorted list.
9. **End**

CODE:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.lang.Double;
class Patient {
    String name;
    String mobileNo;
    String tokenNo;
    String gender;
    String dateOfBirth;
    String address;
    String medicineName;
    double medicineCost;
    int duration;

    public Patient(String name, String mobileNo, String tokenNo, String gender, String
dateOfBirth, String address, String medicineName, double medicineCost, int duration) {
        this.name = name;
        this.mobileNo = mobileNo;
        this.tokenNo = tokenNo;
        this.gender = gender;
        this.dateOfBirth = dateOfBirth;
        this.address = address;
        this.medicineName = medicineName;
        this.medicineCost = medicineCost;
        this.duration = duration;
    }
}
```

```

    public String toString() {
        return "Name: " + name + ", Mobile: " + mobileNo + ", Token No: " + tokenNo + ",
Gender: " + gender + ", DOB: " + dateOfBirth + ", Address: " + address +
        ", Medicine: " + medicineName + ", Cost: $" + medicineCost + ", Duration: " +
duration + " days";
    }
}

public class PatientOperations {
    List<Patient> patients = new ArrayList<>();

    public void loadPatients(String filePath) throws IOException {
        BufferedReader reader = new BufferedReader(new FileReader(filePath));

        String line;
        while ((line = reader.readLine()) != null) {
            String[] details = line.split(",");
            String name = details[0];
            String mobileNo = details[1];
            String tokenNo = details[2];
            String gender = details[3];
            String dateOfBirth = details[4];
            String address = details[5];
            String medicineName = details[6];
            double medicineCost = Double.parseDouble(details[7]);
            int duration = Integer.parseInt(details[8]);

            Patient patient = new Patient(name, mobileNo, tokenNo, gender, dateOfBirth,
address, medicineName, medicineCost, duration);
            patients.add(patient);
        }
        reader.close();
    }

    public void findPatientByName(String name) {
        for (Patient patient : patients) {
            if (patient.name.equalsIgnoreCase(name)) {
                System.out.println(patient);
            }
        }
    }
}

```

```
        return;
    }
}

System.out.println("No patient found with the name " + name);
}public void listPatientsByGender(String gender) {
    boolean found = false;
    for (Patient patient : patients) {
        if (patient.gender.equalsIgnoreCase(gender)) {
            System.out.println(patient);
            found = true;
        }
    }
    if (!found) {
        System.out.println("No patients found with gender " + gender);
    }
}public void listPatientsByMedicineCost(double minCost) {
    for (Patient patient : patients) {
        if (patient.medicineCost > minCost) {
            System.out.println(patient);
        }
    }
}public void listPatientsByMedicine(String medicineName) {
    boolean found = false;
    for (Patient patient : patients) {
        if (patient.medicineName.equalsIgnoreCase(medicineName)) {
            System.out.println(patient);
            found = true;
        }
    }
    if (!found) {
```

```
        System.out.println("No patients found with medicine " + medicineName);
    }
} public static void main(String[] args) {
    PatientOperations operations = new PatientOperations();
    Scanner scanner = new Scanner(System.in);
    try {
        operations.loadPatients("patients.txt");
        while (true) {
            System.out.println("\nChoose an operation:");
            System.out.println("1: Find patient by Name");
            System.out.println("2: List patients by Gender");
            System.out.println("3: List patients with medicine cost above a certain value");
            System.out.println("4: List patients with a certain medicine");
            System.out.println("0: Exit");
            System.out.print("Enter choice: ");
            int choice = scanner.nextInt();
            scanner.nextLine();
            switch (choice) {
                case 1:
                    System.out.print("Enter patient name to search: ");
                    String name = scanner.nextLine();
                    operations.findPatientByName(name);
                    break;
                case 2:
                    System.out.print("Enter gender (Male/Female) to list patients: ");
                    String gender = scanner.nextLine();
                    operations.listPatientsByGender(gender);
                    break;
                case 3:
                    System.out.print("Enter minimum medicine cost: ");
```

```
        double minCost = scanner.nextDouble();
        operations.listPatientsByMedicineCost(minCost);
        break;

    case 4:
        System.out.print("Enter medicine name to list patients: ");
        String medicine = scanner.nextLine();
        operations.listPatientsByMedicine(medicine);
        break;
    case 0:
        System.out.println("Exiting program.");
        scanner.close();
        return;
    default:
        System.out.println("Invalid choice. Please select a valid option.");
    }
}

} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
}
```

OUTPUT:

```
C:\Users\Admin\Desktop\Amirthaa 014>java PatientCollection
Enter details for Patient 1:
Patient ID: 453
Name: Annaya
Disease: Fever
Address: ooty
Phone No: 568986966
Gender (Male/Female/Other): Female
Age: 12
Enter details for Patient 2:
Patient ID: 980
Name: Kayau
Disease: Cough
Address: Nagarcoil
Phone No: 9856458875
Gender (Male/Female/Other): Female
Age: 45
Enter details for Patient 3:
Patient ID: 890
Name: Ramya
Disease: Allergy
Address: Coorg
Phone No: 9077584645
Gender (Male/Female/Other): Female
Age: 23
Enter details for Patient 4:
Patient ID: 458
Name: Tamil
Disease: Simla
```

```
Address: ganga street
Phone No: 958764526
Gender (Male/Female/Other): Female
Age: 25
```

```
Enter gender to list patient details:
Female
```

```
Patients with gender Female:
Patient ID: 453, Name: Annaya, Disease: Fever, Address: ooty, Phone No: 568986966, Gender: Female, Age: 12
Patient ID: 980, Name: Kayau, Disease: Cough, Address: Nagarcoil, Phone No: 9856458875, Gender: Female, Age: 45
Patient ID: 890, Name: Ramya, Disease: Allergy, Address: Coorg, Phone No: 9077584645, Gender: Female, Age: 23
Patient ID: 458, Name: Tamil, Disease: Simla, Address: Erode, Phone No: 88656836565, Gender: Female, Age: 23
Patient ID: 586, Name: Sree, Disease: Fever, Address: ganga street, Phone No: 958764526, Gender: Female, Age: 25
```

```
Enter age to list patients greater than that age:
45
Patients older than 45:
```

```
Enter Patient ID to get patient details:
980
Patient Name: Kayau
```

```
Patients sorted alphabetically by name:
Patient ID: 453, Name: Annaya, Disease: Fever, Address: ooty, Phone No: 568986966, Gender: Female, Age: 12
Patient ID: 980, Name: Kayau, Disease: Cough, Address: Nagarcoil, Phone No: 9856458875, Gender: Female, Age: 45
Patient ID: 890, Name: Ramya, Disease: Allergy, Address: Coorg, Phone No: 9077584645, Gender: Female, Age: 23
Patient ID: 586, Name: Sree, Disease: Fever, Address: ganga street, Phone No: 958764526, Gender: Female, Age: 25
Patient ID: 458, Name: Tamil, Disease: Simla, Address: Erode, Phone No: 88656836565, Gender: Female, Age: 23
```

```
C:\Users\Admin\Desktop\Amirthaa 014>|
```

8.B.**AIM:**

Consider A patient class and assume the necessary fields such as disease or problem, name, address, phno, gender, age, Inpatient or Outpatient, Bill Amount. Consider a collection of Patient Object(Array List) and do the following operation.

- i) list the Patient details of a particular gender
- ii) list the Patient name and disease who are Outpatient
- iii) Display the patients bill amount of a particular patient Name.
- iv) Sort and display the whole patient detail in thier alphabetically order.

PROCEDURE:

1. **Create Patient Class:**
 - Define attributes: patientId, name, disease, address, phoneNo, gender, age, patientType, billAmount.
 - Add a constructor to initialize all attributes.
 - Include getter methods for all attributes and a toString() method for formatted display.
2. **Main Class (PatientCollectionApp):**
 - Use the main method to manage a collection of Patient objects.
3. **Input Patient Details:**
 - Use a loop to input details for 5 patients, including patientType and billAmount.
 - Store each patient in a List<Patient>.
4. **Filter Patients by Gender:**
 - Prompt the user for a gender and display the details of matching patients.
5. **Filter Outpatients:**
 - Loop through the patients and display the names and diseases of patients marked as "Outpatient".
6. **Find Bill Amount by Patient Name:**
 - Ask the user for a patient's name.
 - Search the list and display the corresponding billAmount, or show a "not found" message if no match.
7. **Sort Patients Alphabetically:**
 - Use Collections.sort to sort the list of patients by name.
 - Print the sorted list using their toString() method.
8. **Close Scanner:**
 - Ensure the Scanner object is closed to release resources.

PROGRAM:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
```

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.lang.Double;
class Patient {
    String name;
    String mobileNo;
    String tokenNo;
    String gender;
    String dateOfBirth;
    String address;
    String medicineName;
    double medicineCost;
    int duration;

    public Patient(String name, String mobileNo, String tokenNo, String gender, String
dateOfBirth, String address, String medicineName, double medicineCost, int duration) {

        this.name = name;
        this.mobileNo = mobileNo;
        this.tokenNo = tokenNo;
        this.gender = gender;
        this.dateOfBirth = dateOfBirth;
        this.address = address;
        this.medicineName = medicineName;
        this.medicineCost = medicineCost;
        this.duration = duration;
    }

    public String toString() {

        return "Name: " + name + ", Mobile: " + mobileNo + ", Token No: " + tokenNo + ",
Gender: " + gender + ", DOB: " + dateOfBirth + ", Address: " + address +
        ", Medicine: " + medicineName + ", Cost: $" + medicineCost + ", Duration: " +
duration + " days";
    }
}
```

```

    }
}

public class PatientOperations {
    List<Patient> patients = new ArrayList<>();

    public void loadPatients(String filePath) throws IOException {
        BufferedReader reader = new BufferedReader(new FileReader(filePath));
        String line;
        while ((line = reader.readLine()) != null) {
            String[] details = line.split(",");
            String name = details[0];
            String mobileNo = details[1];
            String tokenNo = details[2];
            String gender = details[3];
            String dateOfBirth = details[4];
            String address = details[5];
            String medicineName = details[6];
            double medicineCost = Double.parseDouble(details[7]);
            int duration = Integer.parseInt(details[8]);

            Patient patient = new Patient(name, mobileNo, tokenNo, gender, dateOfBirth,
            address, medicineName, medicineCost, duration);
            patients.add(patient);
        }
        reader.close();
    }

    public void findPatientByName(String name) {
        for (Patient patient : patients) {
            if (patient.name.equalsIgnoreCase(name)) {
                System.out.println(patient);
                return;
            }
        }
    }

    System.out.println("No patient found with the name " + name);
}

```

```
}public void listPatientsByGender(String gender) {  
    boolean found = false;  
    for (Patient patient : patients) {  
        if (patient.gender.equalsIgnoreCase(gender)) {  
            System.out.println(patient);  
            found = true;  
        }  
    }  
    if (!found) {  
        System.out.println("No patients found with gender " + gender);  
    }  
}  
  
public void listPatientsByMedicineCost(double minCost) {  
    for (Patient patient : patients) {  
        if (patient.medicineCost > minCost) {  
            System.out.println(patient);  
        }  
    }  
}  
  
public void listPatientsByMedicine(String medicineName) {  
    boolean found = false;  
    for (Patient patient : patients) {  
        if (patient.medicineName.equalsIgnoreCase(medicineName)) {  
            System.out.println(patient);  
            found = true;  
        }  
    }  
    if (!found) {  
        System.out.println("No patients found with medicine " + medicineName);  
    }  
}  
  
} public static void main(String[] args) {  
    PatientOperations operations = new PatientOperations();
```

```
Scanner scanner = new Scanner(System.in);

try {
    operations.loadPatients("patients.txt");
    while (true) {
        System.out.println("\nChoose an operation:");
        System.out.println("1: Find patient by Name");
        System.out.println("2: List patients by Gender");
        System.out.println("3: List patients with medicine cost above a certain value");
        System.out.println("4: List patients with a certain medicine");
        System.out.println("0: Exit");
        System.out.print("Enter choice: ");
        int choice = scanner.nextInt();
        scanner.nextLine();
        switch (choice) {
            case 1:
                System.out.print("Enter patient name to search: ");
                String name = scanner.nextLine();
                operations.findPatientByName(name);
                break;
            case 2:
                System.out.print("Enter gender (Male/Female) to list patients: ");
                String gender = scanner.nextLine();
                operations.listPatientsByGender(gender);
                break;
            case 3:
                System.out.print("Enter minimum medicine cost: ");
                double minCost = scanner.nextDouble();
                operations.listPatientsByMedicineCost(minCost);
                break;
            case 4:
```

```

        System.out.print("Enter medicine name to list patients: ");

        String medicine = scanner.nextLine();

        operations.listPatientsByMedicine(medicine);

        break;

    case 0:

        System.out.println("Exiting program.");

        scanner.close();

        return;

    default:

        System.out.println("Invalid choice. Please select a valid option.");

    }

}

} catch (IOException e) {

    System.out.println("Error reading file: " + e.getMessage());

}

}

}

```

OUTPUT:

```

C:\Users\Admin\Desktop\Amirthaa 014>java PatientCollectionApp
Enter details for Patient 1:
Patient ID: 907
Name: Ananya
Disease: Fever
Address: Ooty
Phone No: 895466565
Gender (Male/Female/Other): Female
Age: 44
Patient Type (Inpatient/Outpatient): Outpatient
Bill Amount: 700
Enter details for Patient 2:
Patient ID: 852
Name: Eswar
Disease: Allergy
Address: Thoothukudi
Phone No: 8845568432
Gender (Male/Female/Other): Male
Age: 23
Patient Type (Inpatient/Outpatient): Outpatient
Bill Amount: 445
Enter details for Patient 3:
Patient ID: 4582
Name: Vidhya
Disease: High BP
Address: Ooty
Phone No: 8565163289
Gender (Male/Female/Other): Female

```

Gender (Male/Female/Other): Female
Age: 23
Patient Type (Inpatient/Outpatient): Inpatient
Bill Amount: 890

Enter gender to list patient details:
Female

Patients with gender Female:

Patient ID: 907, Name: Ananya, Disease: Fever, Address: Ooty, Phone No: 895466565, Gender: Female, Age: 44, Patient Type : Outpatient, Bill Amount: Rs 700.0
Patient ID: 4582, Name: Vidhya, Disease: High BP, Address: Ooty, Phone No: 8565163289, Gender: Female, Age: 23, Patient Type: Inpatient, Bill Amount: Rs 890.0

Patients who are Outpatients:

Name: Ananya, Disease: Fever
Name: Eswar, Disease: Allergy

Enter Patient Name to get the bill amount:

Eswar

Bill Amount for Eswar: Rs 445.0

Patients sorted alphabetically by name:

Patient ID: 907, Name: Ananya, Disease: Fever, Address: Ooty, Phone No: 895466565, Gender: Female, Age: 44, Patient Type : Outpatient, Bill Amount: Rs 700.0

Patient ID: 852, Name: Eswar, Disease: Allergy, Address: Thoothukudi, Phone No: 8845568432, Gender: Male, Age: 23, Patient Type: Outpatient, Bill Amount: Rs 445.0

Patient ID: 4582, Name: Vidhya, Disease: High BP, Address: Ooty, Phone No: 8565163289, Gender: Female, Age: 23, Patient Type: Inpatient, Bill Amount: Rs 890.0

C:\Users\Admin\Desktop\Amirthaa 014>

Result:

The Output for the above program has been executed and verified successfully;